[1] Sources: Esri, DeLorme, HERE, MapmyIndia

[2] We omit the implementation here, that will differ from what is shown in chapters 9 and 10 respectively only for a replacing every occurrence of `newPoint` with `newShop.point`, and using `newShop` instead of `newPoint` as argument to the `Node` constructor.

[3] Unless, of course, you also provide IT infrastructure to all the shops as part of the service. Which is possible, although obviously more challenging both technically and financially.

[4] That's probably the easy part, as each shop can write an adapter to bridge the gap between their internal software and your interface.

[5] A race condition is a situation that occurs when a system attempts to perform two or more operations at the same time, but the outcome of the operations depends on them to be executed in a specific order. In other words, a race conditions happen when two operations A and B are executed at the same time, but we get a correct result if, f.i., A finished before B, while we get an error if B finishes before A.

[6] Currying is a functional programming technique that allows to transform the execution of a function with n arguments in a sequence of n executions of unary functions. It allows, for instance, defining a generic function, like `add(a,b)` which adds numbers a and b, and then create new functions by fixing the first argument: `add5 = add(5)`. Or as well just calling the original function like this: `add(4)(3)` to add 4 and 3. Not all programming languages natively allow currying, but many allow workarounds to achieve it.

[7] Stands for `compareShops`: the abbreviation was forced to better fit code lines into the page.

[8] IPFS is a peer-to-peer hypermedia protocol, see https://ipfs.io – it's worth checking it out…

[9] Obviously, if we do get an answer, we can check the HTTP code and see if there is a network error or, assuming the callee correctly implements rules on the HTTP code to send back in case of error, why the remote service is failing.

[10] This is the case with HTTP/1.1, where requests cannot be canceled by the caller. HTTP/2 specification, instead, introduces the possibility to cancel requests sent.

[11] Unfortunately, as mentioned, with HTTP/1.1 we can't decide this timeout, that is set on the servers. We have to adjust to the settings of shops' servers, and in particular to the longest of those timeouts. So, in the example above, assume that 8 seconds is the longest time after which a request to one of these servers is guaranteed to fail.

[12] Meaning that the intensity will be maximum for the red channel, and minimum for the other two channels.

[13] If we store it uncompressed. Formats like JPG or WEBM stores a compressed version of the image with a slight loss in quality but using sensibly less memory, at least one order of magnitude less.

[14] An image at 720p resolution, also known as HD. Today this is even considered poor quality.

[15] Resolution 1080p or full HD. Still, far from the 4K resolution that's the standard these days.